

CSC3400 Database Principles

Chapter 8 & 9

Stored Procedure & Trigger

Learning Objective

Explain why would we need to use procedural code.

Explain the relationship between procedural code and data triggers.

Explain how to use data triggers to make change automatically.

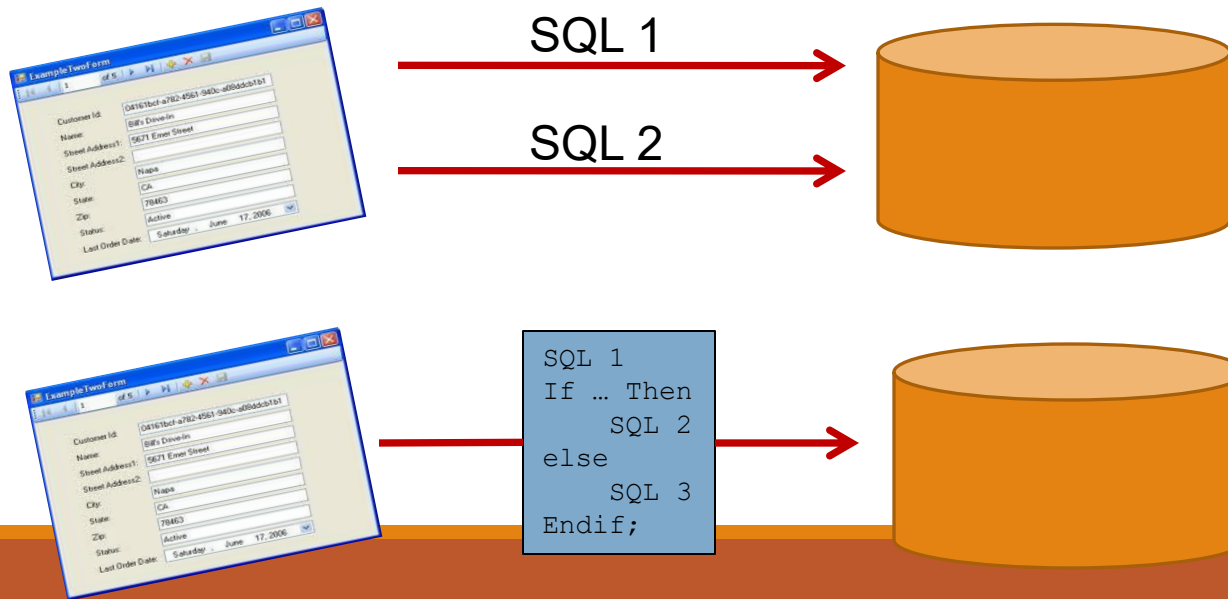
Introduction

- In business applications often multiple changes need to be made together.
- Such situation need to be handled correctly to ensure the integrity of data.
- SQL commands are powerful, however in many situations require the ability to execute selected multiple statements.
- Although, there are diverse methods to implement procedural languages, it is helpful when the language is embedded into the query system.
- With this approach, all of the code and conditions remain within the database definition and constraints are enforced automatically for all applications.
- These conditions are often written as data triggers (e.g. code that is executed when some data element is modified).

Procedural Language

Procedural language is a traditional programming language to specify the sequence of a set of commands.

SQL commands only tell what to do not how to do it.



Where Should Code be Located?

It can be located in one of the following locations:

- Within the query system
- Within forms and reports
- In external programs

In general, code that relates to data should be placed within the query system.

Reasons:

- To be written only once instead of copied into multiple forms
- The DBMS will ensure that the code is always executed and not bypassed.

Where Should Code be Located? *..cont.*

For example, think about a security situation.

Every time someone changes an employee salary, need to provide log information in a log table.

What happen if it is implemented within forms?

- Forget to include the code in new forms
- Change the salary directly using SQL commands.

Therefore, placing the code in the query system provides a mechanism to ensure that the code is run anytime the data is changed, regardless how the modification is done.

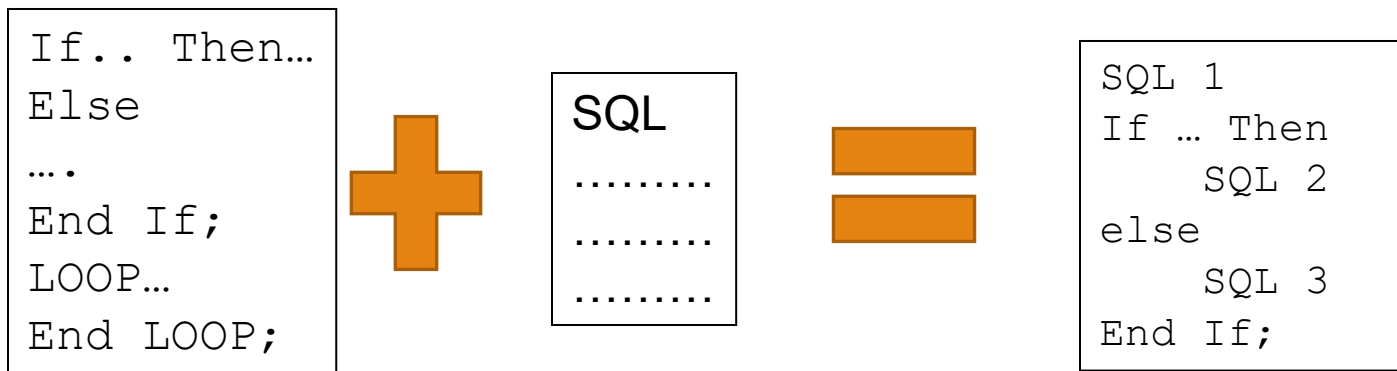
PL/SQL

Stands for “Procedural Language extension to SQL”

Oracle Corporation Standard data access language for relational databases.

Seamlessly integrates procedural constructs with SQL.

Use for creating triggers and stored procedures.



PL/SQL Block Structure

DECLARE (optional)

- Variables, cursors, user-defined exceptions

BEGIN (mandatory)

- SQL statements
- PL/SQL statements

EXCEPTION (optional)

- Action to perform when exceptions occurs

END; (mandatory)

PL/SQL Block Structure *..cont.*

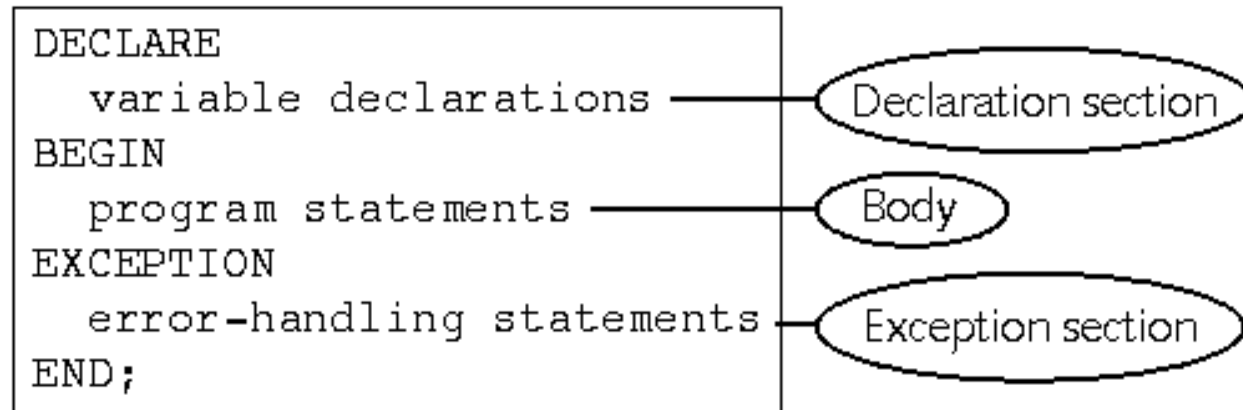


Figure 4-1 Structure of a PL/SQL program block

Comments:

- Not executed by interpreter
- Enclosed between `/*` and `*/`
- On one line beginning with `--`

PL/SQL Example

Predefined
Oracle
Procedure

```
--PL/SQL program to display the current date
DECLARE
    todays_date DATE;
BEGIN
    todays_date :=SYSDATE;
    DBMS_OUTPUT.PUT_LINE('Today`s date is `');
    DBMS_OUTPUT.PUT_LINE(todays_date);
END;
```



To enable output in SQL Developer, execute the following command before running the PL/SQL program.

```
SET SERVEROUTPUT ON
```

Another Example

```
--PL/SQL program to display first name of empid = 100
DECLARE
    v_fname VARCHAR2(20);
BEGIN
    SELECT fname
    INTO v_fname
    FROM employee
    WHERE empid = 100;
    DBMS_OUTPUT.PUT_LINE('The first name of empid=100 is ' || v_fname);
END;
```

Output:

The first name of empid=100 is *<the value of v_fname>*

PL/SQL Example

```
declare
    cursor emp_cursor is select * from employee ;
    emp_val emp_cursor%ROWTYPE ;
    cnt    INTEGER(5) := 0 ;
begin
    open emp_cursor ;
    loop
        fetch emp_cursor into emp_val ;
        exit when emp_cursor%NOTFOUND ;
        cnt := cnt + 1 ;
    end loop ;
    close emp_cursor ;
    if (cnt > 0)
    then insert into total_employee values(cnt) ;
    end if ;
end ;
/
```

PL/SQL

Cursor properties

- %FOUND, %NOTFOUND : a record can/cannot be fetched from the cursor
- %ISOPEN : the cursor has been opened
- %ROWCOUNT : the number of rows fetched from the cursor so far
- %TYPE : the data type of a column:

Conditional logic

```
IF <condition> THEN
    <statements>
ELSIF <condition> THEN
    <statements>
ELSE
    <statements>
END IF;
```

Declaring Variable with %TYPE attribute

To be used when declaring a variable according to:

- ❑ A database column definition
- ❑ Previously declared variable

Is prefix with:

- ❑ The database table and column name
- ❑ The declared variable.

Example with %TYPE attribute

```
--PL/SQL program to display first name of empid=100  
  
DECLARE  
  
    v_salary number (7,2);  
    v_fname employee.fname%TYPE;  
    v_minsalary v_salary%TYPE;  
  
BEGIN  
  
    SELECT fname  
    INTO v_fname  
    FROM employee  
    WHERE empid = 100;  
  
    DBMS_OUTPUT.PUT_LINE('The first name of Employee 100 is ' ||  
v_fname);  
  
END;
```

PL/SQL – Loops

For loop:

```
for item_val in item_cursor  
loop  
end loop ;
```

//note no need for open/fetch/close cursor commands
in this case.

While loop:

```
while cnt < 5  
loop  
end loop ;
```


Example PL/SQL: LOOP

```
declare
    cursor item_cursor is select I.iid, I.oid, count(B.bid) bidcnt
                           from items I, bids B where I.iid = B.iid
                           group by I.iid, I.oid ;

    item_val item_cursor%ROWTYPE ;
begin
    update owners set status = 'B' ;

    for item_val in item_cursor
    loop
        if (item_val.bidcnt > 2) then
            update owners set status = 'G'
                where oid = item_val.oid;
        end if;
    end loop ;
end ;

/
```

PL/SQL – exceptions

Exceptions arise when a statement causes an error.

Examples:

`ACCESS_INTO_NULL`

`CURSOR_ALREADY_OPEN`

`DUP_VAL_ON_INDEX`

`INVALID_CURSOR`

`LOGIN_DENIED`

`NO_DATA_FOUND`

`NOT_LOGGED_ON`

`TOO_MANY_ROWS` – select into stmt returns more than one row

`VALUE_ERROR`

`ZERO_DIVIDE`

Using PL/SQL to write Simple Procedure

The syntax of procedure:

Creating *add_dept* procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(argument1 datatype1,...)]
IS | AS
procedure_body;
```

```
CREATE PROCEDURE add_dept IS
    v_dept_id      dept_temp.deptt_id%TYPE;
    v_dept_name     dept_temp.deptt_name%TYPE;
BEGIN
    v_dept_id := 20;
    v_dept_name := 'Department of Multimedia';
    INSERT INTO dept_temp(deptt_id, deptt_name)
    VALUES (v_dept_id, v_dept_name);
    DBMS_OUTPUT.PUT_LINE(' Inserted ' || SQL%ROWCOUNT || ' row ');
END;
```

Using PL/SQL to write Simple Procedure *..cont.*

Invoking the add_dept procedure:

```
... .  
BEGIN  
    add_dept;  
END;
```

Output:

Inserted *<the value SQL%ROWCOUNT>* row

What is Trigger?

A trigger is an even-condition-action rule

- When an event occurs, test condition; if condition satisfied, execute action

Example:

- Event: whenever there come a new student..
- Condition: with CGPA higher than 3.75
- Action: then include his/her name in Dean's list

Trigger Syntax

```
CREATE [OR REPLACE] TRIGGER name
    {BEFORE | AFTER | INSTEAD OF}
    {DELETE
    | INSERT
    | UPDATE [OF column [,column] ...]}
    [OR
    {DELETE
    | INSERT
    | UPDATE [OF column [,column]
                ...]}] ...
ON {TABLE | VIEW} tablename
[ [REFERENCING {OLD [AS] old
                | NEW [AS] new} ...]
FOR EACH {ROW | STATEMENT}
[WHEN (condition)] ]
PL/SQL block
```

Trigger Example

```
CREATE TRIGGER DeansList
```

```
AFTER INSERT ON Student
```

← **Event**

```
REFERENCING NEW AS newStudent
```

```
FOR EACH ROW
```

← **Condition**

```
WHEN (newStudent.GPA > 3.0)
```

```
INSERT INTO Dean_List VALUES  
(:newStudent.studentID, :newStudent.studentName, newStudent.GPA);
```

↑ **Action**

Another Example

```
CREATE OR REPLACE
TRIGGER trigger_1
  BEFORE UPDATE OF salary ON employee
  FOR EACH ROW
  BEGIN
    UPDATE total
      SET sal = sal + :new.salary
                        - :old.salary;
  END;
/
```

Assume this table exists.
One row, one column

total
sal
1,234,567

Trigger Options

Possible events include:

- INSERT ON <table>
- DELETE ON <table>
- UPDATE [OF <column>] ON <table>

Trigger can be activated:

- FOR EACH ROW modified
- FOR EACH STATEMENT that performs modification

Action can be executed:

- AFTER or BEFORE the triggering event

Transition Variables

OLD ROW – the modified row before the triggering event.

NEW ROW – the modified row after the triggering event.

OLD TABLE – a hypothetical read-only containing all modified rows before the triggering event.

NEW TABLE – a hypothetical table containing all modified rows after the triggering event.

Example: Before Trigger

Check that
enrollment $\leq$ limit

```
CREATE TRIGGER Max_EnrollCheck
  BEFORE INSERT ON Transcript
    REFERENCEING NEW AS N    --row to be added
  FOR EACH ROW
  WHEN
    ((SELECT COUNT (T.StudId) FROM Transcript T
      WHERE T.CrsCode = N.CrsCode
          AND T.Semester = N.Semester)
    >=
    (SELECT C.MaxEnroll FROM Course C
      WHERE C.CrsCode = N.CrsCode ))
  ABORT TRANSACTION
```

Example: After Trigger

```
CREATE TRIGGER LimitSalaryRaise
  AFTER UPDATE OF Salary ON Employee
  REFERENCING OLD AS O
                NEW AS N
  FOR EACH ROW
  WHEN (N.Salary - O.Salary > 0.05 * O.Salary)
    UPDATE Employee
    SET Salary = 1.05 * O.Salary
    WHERE Id = O.Id
```

No salary raises
greater than 5%

Example: After Trigger

Keep track of salary averages in the log

```
CREATE TRIGGER RecordNewAverage
  AFTER UPDATE OF Salary ON Employee
  FOR EACH STATEMENT
    INSERT INTO Log
    VALUES (CURRENT_DATE,
            SELECT AVG (Salary)
            FROM Employee)
```

Triggers can call Procedures

```
CREATE OR REPLACE PROCEDURE sumit
AS
    sal_sum employee.salary%TYPE;
BEGIN
    SELECT sum(salary)
    into sal_sum
    FROM employee;
    UPDATE total
    SET sal = sal_sum;
END;
```

```
CREATE OR REPLACE TRIGGER myTrigger
AFTER UPDATE OF salary ON employee
BEGIN
    sumit;
END;
```

Another Example: Trigger calls Procedure

A trigger to compare an employee's salary to his/her supervisor during insert or update operations:

```
CREATE TRIGGER INFORM_SUPERVISOR
BEFORE INSERT OR UPDATE OF
    SALARY, SUPERVISOR_SSN ON EMPLOYEE
FOR EACH ROW
WHEN
    (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE
                    WHERE
SSN=NEW.SUPERVISOR_SSN) )
    INFORM_SUPERVISOR (NEW.SUPERVISOR_SSN, NEW.S
SN) ;
```

INFORM_SUPERVISOR is a **stored procedure**.

Designing Triggers

Use triggers to guarantee that when a specific operation is performed, related actions are performed.

Do not define triggers that duplicate the functionality already built into Oracle. For example, do not define triggers to enforce data integrity rules that can be easily enforced using declarative integrity constraints.

Limit the size of triggers. If the logic for your trigger requires much more than 60 lines of PL/SQL code, then it is better to include most of the code in a stored procedure and call the procedure from the trigger.

Designing Triggers *..cont.*

Use triggers only for centralized, global operations that should be fired for the triggering statement, regardless of which user or database application issues the statement.

Do not create recursive triggers. For example, creating an AFTER UPDATE statement trigger on the Emp_tab table that itself issues an UPDATE statement on Emp_tab, causes the trigger to fire recursively until it has run out of memory.

Use triggers on DATABASE judiciously. They are executed for *every* user *every* time the event occurs on which the trigger is created.

Summary

Today lesson you have learned how to:

- Explain why would we need to use procedural code.
- Explain the relationship between procedural code and data triggers.
- Explain how to use data triggers to make change automatically.